

Lezione 1 : “Hello World”

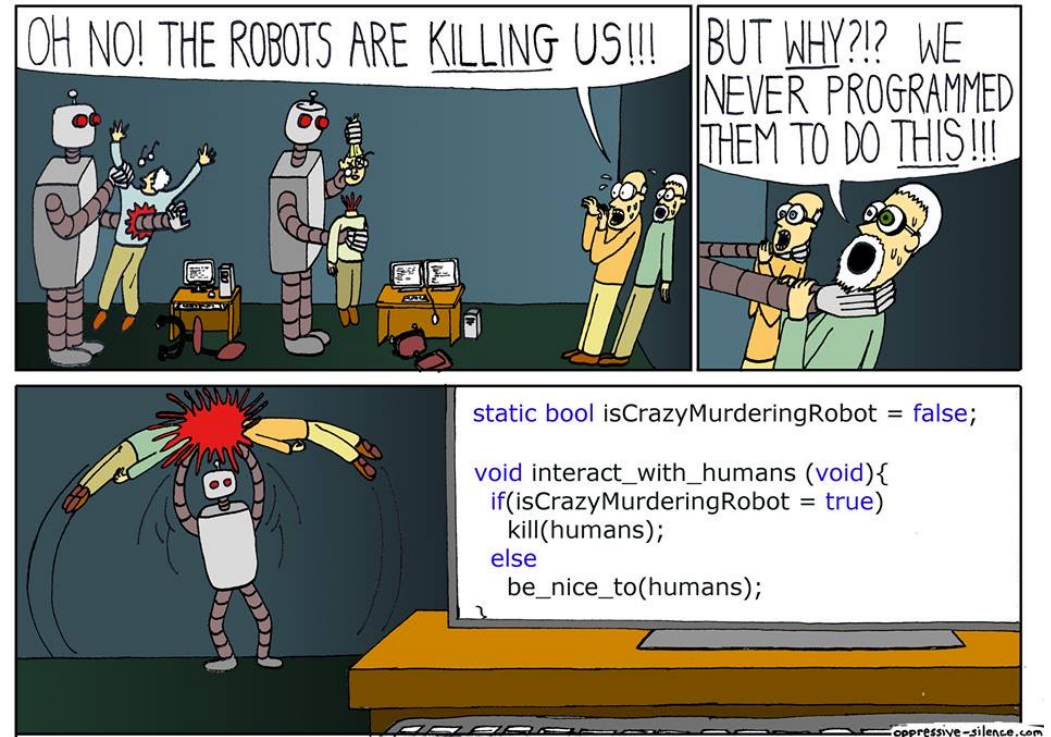
L. Carminati
Università degli Studi di Milano

Laboratorio Trattamento Numerico dei Dati Sperimentali

Obiettivi del corso

❑ Approfondire l'uso di un **linguaggio di programmazione** "ricco" e completo (C++)

- ❑ Riprendere elementi di base del linguaggio C : variabili, strutture di controllo, funzioni, algoritmi...
- ❑ Introdurre elementi di C++ (programmazione ad oggetti) : classi, ereditarietà, polimorfismo.
- ❑ Utilizzo librerie standard (contenitori e algoritmi)



❑ Utilizzare la programmazione per **risolvere problemi** : l'approccio numerico e' molto spesso l'unica soluzione

- ❑ Analisi di dati sperimentali (medie, varianze, istogrammi, correlazione)
- ❑ Algoritmi : ricerca zeri, risoluzione numerica di integrali ed equazioni differenziali
- ❑ Tecniche Monte Carlo per simulazione di esperimenti e risoluzione di integrali

Dettaglio argomenti

	Quite boring
	Interesting
	Real fun

Lezione	Argomento	Linguaggio
1	Analisi set di dati (media, varianza, mediana)	Basics, allocazione dinamica
2	Analisi set di dati (media, varianza, mediana)	Classi in C++
3	Analisi set di dati (media, varianza, mediana)	Contenitori STL e ROOT
facoltativa	Esempi di analisi dati	
4	Campo di dipolo / Campo gravitazionale	Ereditarieta' / incapsulamento
5	Campo di dipolo / Campo gravitazionale	Ereditarieta' / incapsulamento
6	Ricerca zeri di una funzione	Polimorfismo
7	Quadratura numerica	
8	Equazioni differenziali: oscillatori armonici, moto in campo gravitazionale, moto particella carica in campo elettrico e magnetico	Overloading operatori matematici
9	Equazioni differenziali: oscillatori armonici, moto in campo gravitazionale, moto particella carica in campo elettrico e magnetico	
10	Generatori di numeri casuali, integrazione MC	
11	Generatori di numeri casuali, integrazione MC	
12	Simulazione di apparati sperimentali	

Calendario indicativo

		Teoria	Lunedì pomeriggio	Martedì pomeriggio	Mercoledì pomeriggio	Giovedì pomeriggio	Venerdì mattina
			T1	T2	T3	T4	T5
			Carminati	Tomasi	Galli	Maino	Carminati
			14:30-17:30	14:30-17:30	14:30-17:30	14:30-17:30	8:30-11:30
Lezione 1	Arrays	25/09/2023	25/09/2023	26/09/2023	27/09/2023	28/09/2023	29/09/2023
Lezione 2	Vettore	02/10/2023	02/10/2023	03/10/2023	04/10/2023	05/10/2023	06/10/2023
Lezione 3	Template e vectors	09/10/2023	09/10/2023	10/10/2023	11/10/2023	12/10/2023	13/10/2023
Lezione 4	Classi ed ereditarietà	16/10/2023	16/10/2023	17/10/2023	18/10/2023	19/10/2023	20/10/2023
Lezione 5	Classi ed ereditarietà	23/10/2023	23/10/2023	24/10/2023	25/10/2023	26/10/2023	27/10/2023
Lezione 6	Classi astratte e ricerca zeri	30/10/2023	30/10/2023	31/10/2023	08/11/2023	02/11/2023	03/11/2023
Lezione 7	quadratura numerica	06/11/2023	06/11/2023	07/11/2023	15/11/2023	09/11/2023	10/11/2023
Lezione 8	equazioni differenziali	13/11/2023	13/11/2023	14/11/2023	22/11/2023	16/11/2023	17/11/2023
Lezione 9	equazioni differenziali	20/11/2023	20/11/2023	21/11/2023	29/11/2023	23/11/2023	24/11/2023
Lezione 10	generatori numeri casuali	27/11/2023	27/11/2023	28/11/2023	06/12/2023	30/11/2023	01/12/2023
Lezione 11	integrazione MC	04/12/2023	04/12/2023	05/12/2023	13/12/2023	14/12/2023	15/12/2023
Lezione 12	metodi montecarlo	11/12/2023	11/12/2023	12/12/2023	20/12/2023	21/12/2023	22/12/2023

- ❑ Se non ci sono incidenti di percorso possiamo finire il laboratorio entro le vacanze di Natale
- ❑ I turni preliminari sono disponibili al seguente link:
 - ❑ https://docs.google.com/spreadsheets/d/1Ju9isGpp_pgPd7Ttv2Mf8_vF3f5252vAJge_qFZmKpuA/edit?usp=sharing
 - ❑ Non esitate a segnalarmi errori, inesattezze o dimenticanze
 - ❑ Comunicatemi se intendete seguire il corso serale

Technicalities

- ❑ L'insegnamento 2023/2024 si terrà in **presenza**
 - ❑ Lezione di teoria per tutti 12.45-14.15 (?) propedeutica alla sessione pratica di laboratorio. Verrà svolta in presenza in aula A (possibile trasmissione sincrona). No intervallo?
 - ❑ Per le sessioni di laboratorio NON è prevista la connessione remota
 - ❑ In caso di problemi seri è possibile (previo accordo con il docente) recuperare una sessione di laboratorio in un turno diverso da quello assegnato
- ❑ Riferimento sito ARIEL: comunicazioni inviate attraverso quel canale
<https://ltns.ariel.ctu.unimi.it/v5/home/Default.aspx>
- ❑ Lezioni guidate da tenere come riferimento per le sessioni di laboratorio
<http://labmaster.mi.infn.it/Laboratorio2/labTNS/> e <https://labtns.docs.cern.ch/>
- ❑ Ricevimento libero dopo lezione (anche prima o dopo le sessioni di lab) oppure su appuntamento per questioni più complesse
- ❑ Bibliografia verrà discussa a lezione: dispense e manuali standard di C++ (il libro di testo che avete va benissimo, ottime dispense in rete) e analisi statistica
 - ❑ [Stackoverflow](#) la vostra luce in luoghi oscuri quando ogni altra luce si spegne

Organizzazione pratica

❑ Lezioni pratiche di programmazione:

❑ 5 turni di laboratorio (il docente di riferimento per turno e' indicativo), sincronizzati con quelli del laboratorio di fisica: inizio puntuale (no quarto d'ora accademico salvo diversi accordi con il docente di riferimento)

- ❑ Lunedì : 14.30-17.30 (Carminati)
- ❑ Martedì : 14.30-17.30 (Tomasì)
- ❑ Mercoledì : 14.30-17.30 (Galli)
- ❑ Giovedì : 14.30-17.30 (Maino)
- ❑ Venerdì : 8.30-11.30 (Carminati)

❑ 12 lezioni da 3 ore.

❑ Materiale didattico necessario sul sito web del laboratorio.

❑ Esercizi svolti in parte in modo guidato, in parte autonomo

❑ Si può programmare dalle macchine del laboratorio o dal proprio PC personale

❑ La presenza al laboratorio e' obbligatoria : verranno controllate le presenze (!)

Technicalities

- ❑ Non ci sono propedeuticità rispetto al corso di informatica.
 - ❑ L'inizio dovrebbe essere abbastanza soft, C procedurale
 - ❑ Si assume però che si conoscano i tipi di variabile e le principali strutture di controllo
- ❑ Modalità esame :
 1. Consegna degli esercizi (programmi) assegnati durante il laboratorio
 2. Esercizio di programmazione in laboratorio (da svolgere singolarmente) o sempre via replit in funzione dell'andamento dell'emergenza sanitaria
 3. Esame orale :
 - ❑ Discussione linea per linea di un vostro codice, discussione caratteristiche del linguaggio, costruzione di un programma al volo
 - ❑ A partire da un vostro codice discutere l'algoritmo implementato e le sue proprietà e in generale gli argomenti sviluppati nelle lezioni teoriche

Sappiamo bene che è faticoso per via del carico generale di lavoro che avrete ma cercate di seguire regolarmente, terminate le esercitazioni a casa, tartassare docenti/assistenti e fare subito l'esame !

Esempio di test finale

Funzione integrale e' un for() loop :
fisso un valore di x e calcolo l'integrale.

Esercizio 1

L'ampiezza della figura di diffrazione generata da luce di lunghezza d'onda λ attraverso una fenditura di apertura $d=80 \text{ }\mu\text{m}$ nella posizione x su uno schermo distante $L=1 \text{ m}$ è data da un integrale del tipo:

$$A(x) = \int_{-d/2}^{d/2} dt \frac{1}{d} \cos \left[\frac{L}{\lambda} \left(\sqrt{1 + \left(\frac{x-t}{L} \right)^2} - \sqrt{1 + \frac{x^2}{L^2}} \right) \right]$$

1. Data una lunghezza d'onda di $\lambda=500 \text{ nm}$ fare un grafico di $A(x)$ per $-10 \text{ cm} < x < 10 \text{ cm}$.
 2. Calcolare il valore più basso di $|x|$ per cui l'ampiezza è nulla.
 3. Ripetere il punto precedente per $\lambda=400 \text{ nm}$ e 450 nm .
- Calcolare gli integrali con il metodo dei trapezoidi ed un errore assoluto di almeno 10^{-4} .

Esercizio 2

Si implementi una classe in grado di generare numeri casuali distribuiti secondo la densità di probabilità $g(x) = \frac{1}{2} \sin x$ con x nell'intervallo $[0, \pi]$:

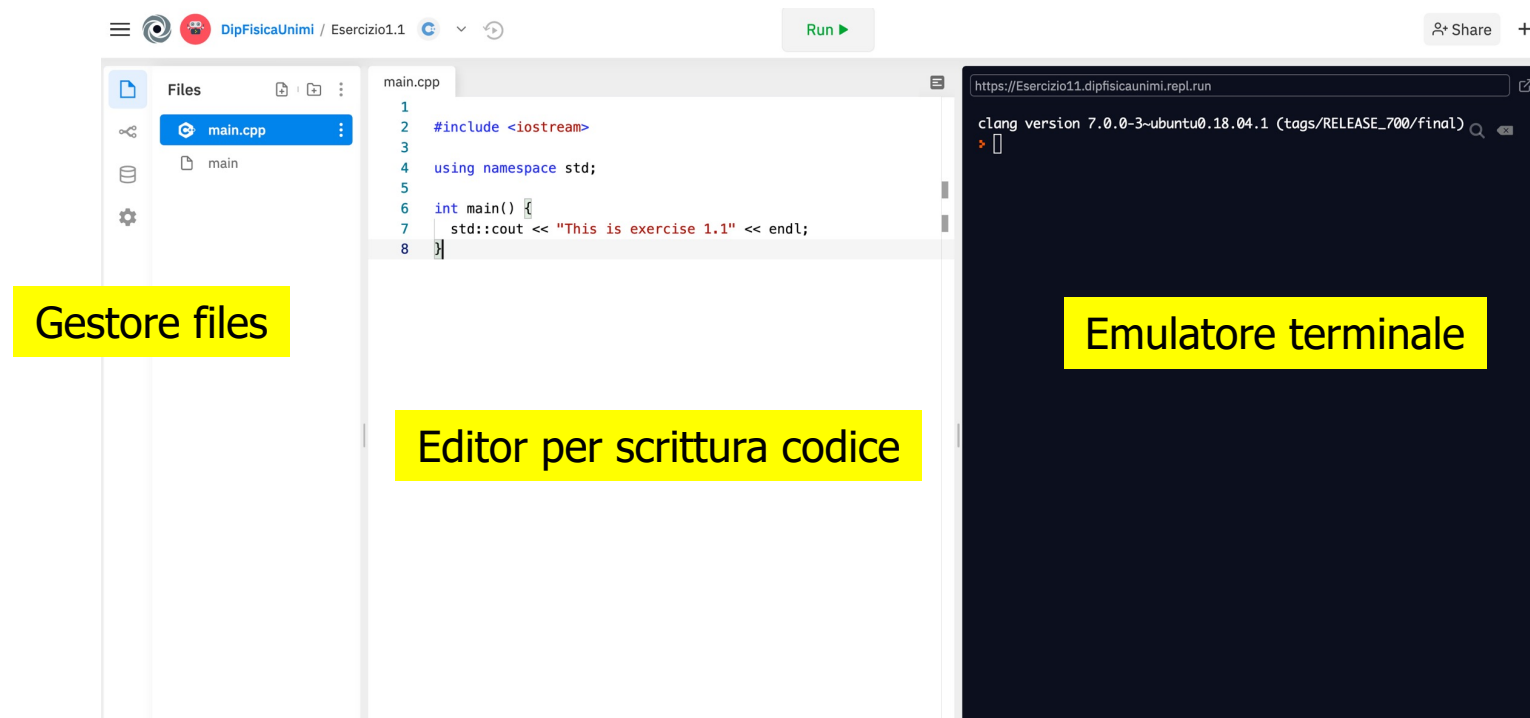
1. con il metodo *accept-reject*;
2. con il metodo della trasformata.

Quest'ultimo metodo si basa sul fatto che se r è una variabile casuale distribuita uniformemente in $[0,1]$ e data la funzione $F(\xi) = \int_{x_{\min}}^{\xi} g(t) dt$, allora $x = F^{-1}(r)$ segue la distribuzione $g(x)$.

In entrambi i casi, produrre un istogramma di valori generati.

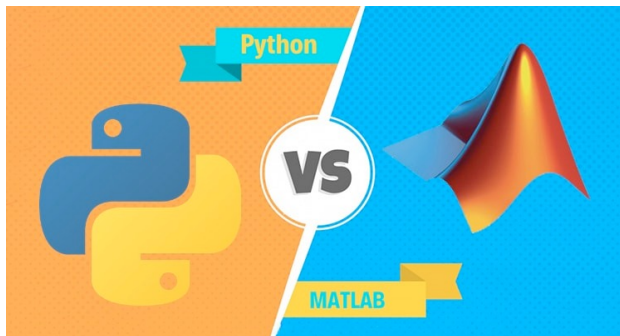
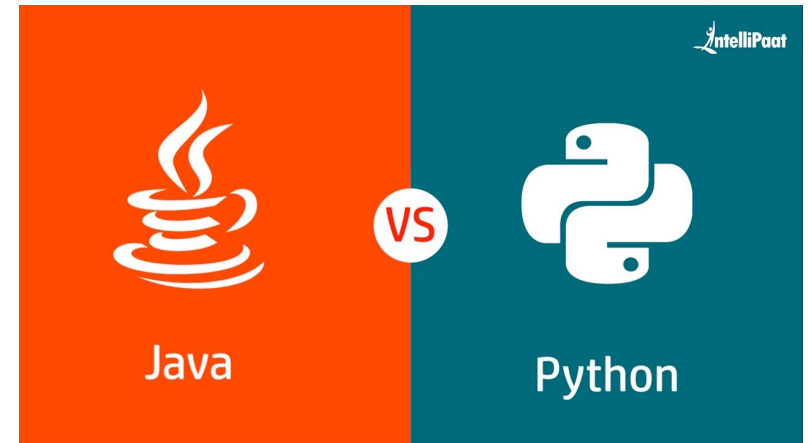
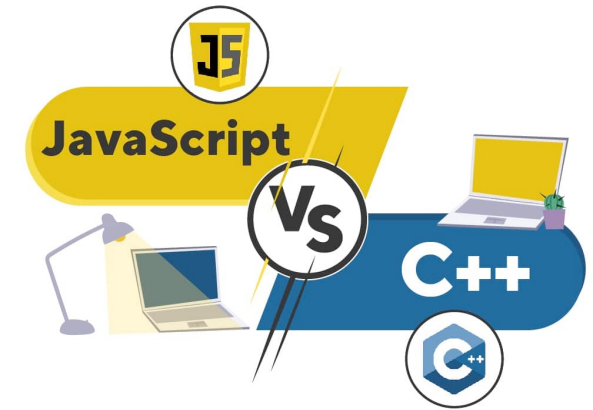
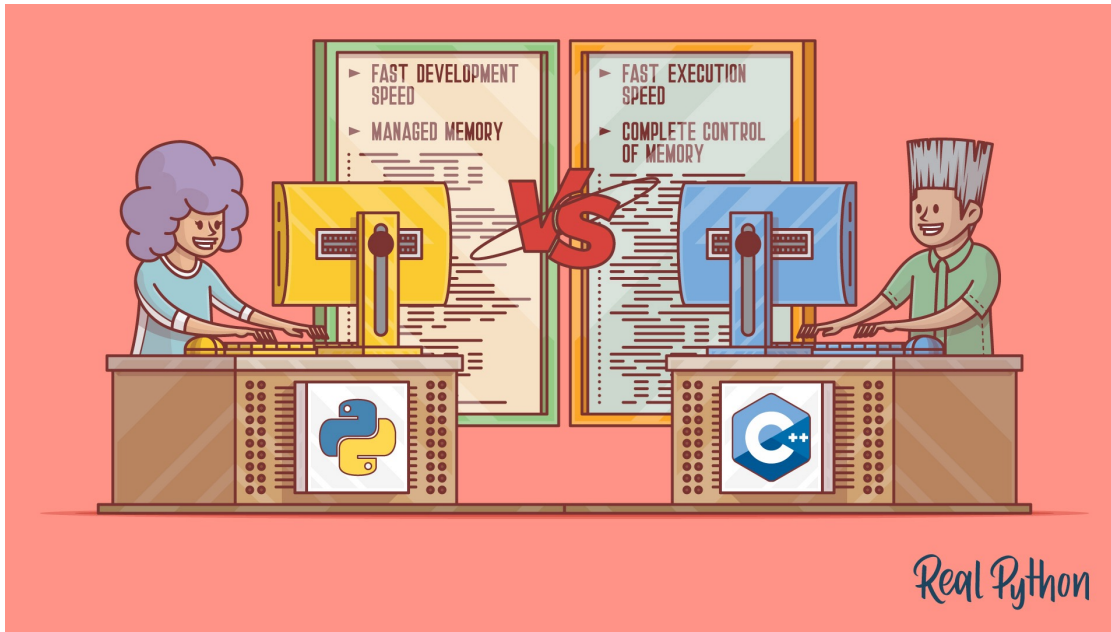
Technicalities

- ❑ Proponiamo ancora l'utilizzo della piattaforma <https://replit.com/>
 - ❑ Permette di lavorare senza avere un setup specifico sulla propria macchina, basta connessione internet e browser: molto comodo ma a volte ha problemi di 'robustness'
 - ❑ Potete continuare a lavorare sui vostri laptops o sulle macchine del laboratorio



- ❑ Registratevi a replit.com: CognomeZZZ (ZZZ ultime tre cifre numero di matricola)
- ❑ Trovate link di adesione al team relativo al vostro turno nel googledoc

Obiettivi del corso : perche' C++ ?



Choose the right tool for your job !

Lezione 1 : analisi dati con arrays del C

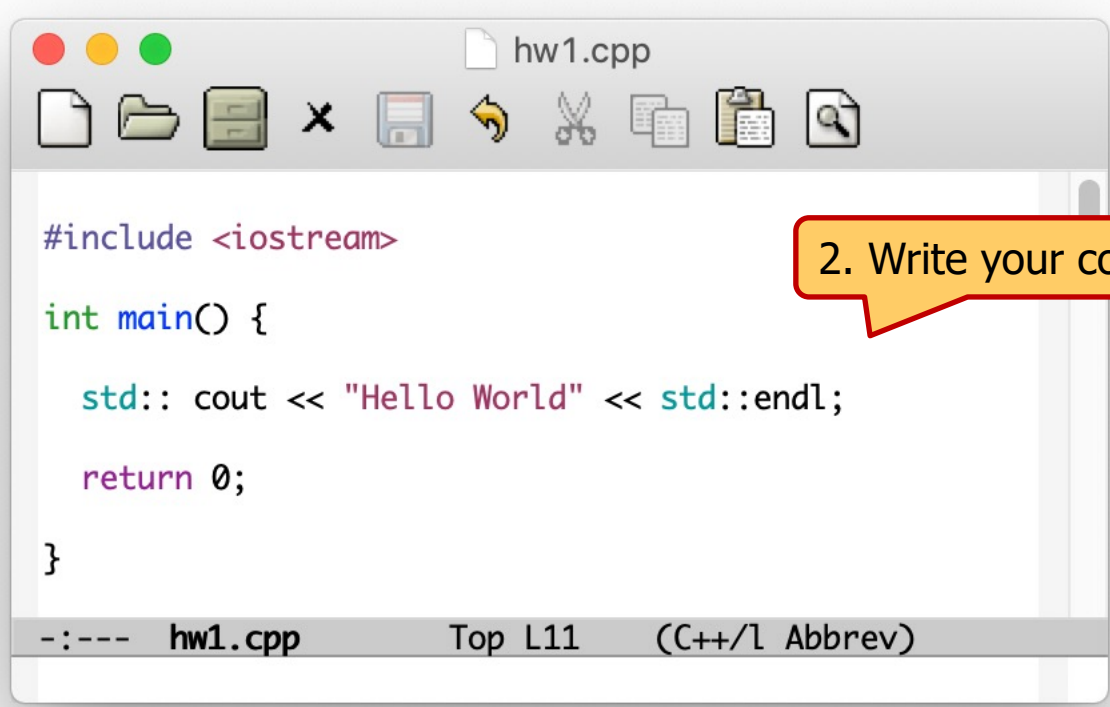
<https://labtnnds.docs.cern.ch/Lezione1/Lezione1/>

- ❑ Tipo di dato da leggere è costituito da numeri double.
- ❑ Tipo di contenitore di dati è un array (dinamico) del C.
- ❑ Operazioni sui dati vengono svolte mediante funzioni.

Straight to the point (no-frills) : my first program

```
> gedit hw1.cpp &
```

1. Open your favourite editor (or open your replit project)



```
#include <iostream>

int main() {

    std::cout << "Hello World" << std::endl;

    return 0;

}
```

The screenshot shows a code editor window with a title bar 'hw1.cpp'. The editor contains the following C++ code: `#include <iostream>`, `int main() {`, `std::cout << "Hello World" << std::endl;`, `return 0;`, and `}`. The status bar at the bottom indicates the file is 'hw1.cpp', the cursor is at 'Top L11', and the language is '(C++/l Abbrev)'.

2. Write your code

```
> g++ hw1.cpp -o hw1
> ./hw1
Hello world
```

3. Compile your code invoking g++

4. Execute your code

1. My first program : details

1. You shall consider your code as a function with some arguments as inputs (none in this case) and returning something (an integer 0 in this case)

2. Include iostream library (io = input/output) : typically to read from standard input (keyboard) and write to standard output (video)

```
#include <iostream>

int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

5. Hey what's this ? It's a namespace (can you wait for a few slides ?)

3. Your program must return something (typically 0 means "all ok")

4. cout is connected with the standard output (video) [cin is connected with the standard input (keyboard)]

2. Standard input/output

```
#include <iostream>

int main() {

    double a , b ;
    std::cout << "Input first number" << std::endl;
    std::cin >> a ;
    std::cout << "Input second number" << std::endl;
    std::cin >> b ;
    double c = a + b ;

    std::cout << "La somma vale : " << c << std::endl;

}
```

Here I declare two variables of type double

Read a value from the standard input through cin

One instruction, three effects :
1. declare a variable of type double ("c")
2. Compute the sum of "a" and "b"
3. Initialize c as the sum of two variables ("a","b")

No return statement, assume 0 by default

Send the output of the calculation to the standard output (video)

3. Variables types

```
#include <iostream>

int main() {

    int n ;
    unsigned int k = 0;
    double a = 1;
    double b = 2;
    double c = a + b ;

    double * address = &a ;
    std::cout << address << " " << *address << std::endl;

    char nome[10] = "foo" ;

    std::cout << "La somma vale : " << c << std::endl;
    std::cout << "Il contenuto della variabile nome : " << nome << std::endl;

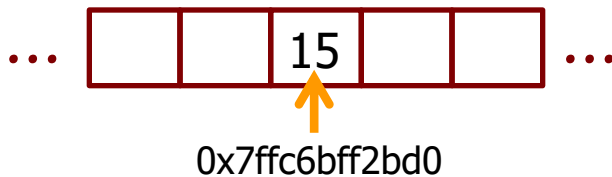
}
```

A few variables types

OMG pointers !

"nome" can contain up to 10 characters

3. Variables types : pointers



- ❑ (double *) : this is the variable type. "addressA is a variable of type (double *), a variable built to host an address to double
- ❑ &a : & is the reference operator, returns the address of a variable

The address of variable "a" (&a) is deposited into an address variable called "address"

```
double a = 15 ;  
double * address = & a ;  
  
std::cout << "Valore contenuto in a" << a << std::endl;  
std::cout << "L'indirizzo della variabile a" << address << std::endl;  
std::cout << "Valore contenuto in a" << *address << std::endl;
```

An arrow points from the variable 'address' in the code to the text box below.

de-referentiation operator (*) : when applied to a pointer returns the content

```
Valore contenuto in a      15  
L'indirizzo della variabile a 0x7ffc6bff2bd0  
Valore contenuto in a      15
```

- ❑ & (variable) returns the address of a variable
- ❑ * (pointer) returns the variable pointed by the address

4. Control structures

```
#include <iostream>

using namespace std;

int main() {

    int k = 0;

    if ( k < 10 ) {
        cout << "k is smaller than 10" << endl;
    } else {
        cout << "k is larger than 10" << endl;
    };

    for ( int k = 0 ; k < 10 ; k++ ) {
        cout << "k = " << k << end;
    };

    while ( k < 10 ) {
        cout << "k = " << k << end;
        k++;
    }

}
```

Selection statement : if (condition) {} else {}

Iteration statement : the for loop

Iteration statement : the while loop (check also also the do{} while () flavor)

5. Functions : structuring a bit the code

Declare functions :

`<return_type> <function name> (<input1>,<input2>.. )`

```
#include <iostream>

double sum ( double a, double b) {
    return a+b;
};

int main() {

    double a , b ;
    std::cout << "Input first number" << std::endl;
    std::cin >> a ;
    std::cout << "Input second number" << std::endl;
    std::cin >> b ;
    double c = sum(a,b) ;
    std::cout << "La somma vale : " << c << std::endl;

}
```

Call your function :
sum returns a double
which is deposited
into the variable c

Organising the code in functions is crucial: always create functions when possible !

- ❑ Simplify the main program, a function can be called several times in the main
- ❑ A function can be saved in a separate file and re-used in different projects

5. Functions : connections with the main

```
#include <iostream>

void scambiaByValue( double a, double b) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByReference( double &a , double &b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByPointer( double* a , double* b) {
    double c=*a;
    *a=*b;
    *b=c;
}

int main() {
    double a = 5 ;
    double b = 4 ;
    std::cout << a << " " << b << std::endl;
    scambiaByValue(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByReference(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByPointer(&a,&b) ;
}
```

Let's write functions to exchange the content of two variables.

❑ Functions can be placed before the main

5. Functions : connections with the main

```
#include <iostream>

void scambiaByValue( double a, double b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByReference( double &a , double &b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByPointer( double* a , double* b ) {
    double c=*a;
    *a=*b;
    *b=c;
}

int main() {
    double a = 5 ;
    double b = 4 ;
    std::cout << a << " " << b << std::endl;
    scambiaByValue(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByReference(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByPointer(&a,&b) ;
}
```

Let's write functions to exchange the content of two variables.

☐ Functions can be placed before the main

- Passing by value : when the function is called a local copy of a and b is built (no relation with the variables a and b in the main).

- ☐ Effect on a and b variables in the function
- ☐ No effect on the variables a and b in the main

5. Functions : connections with the main

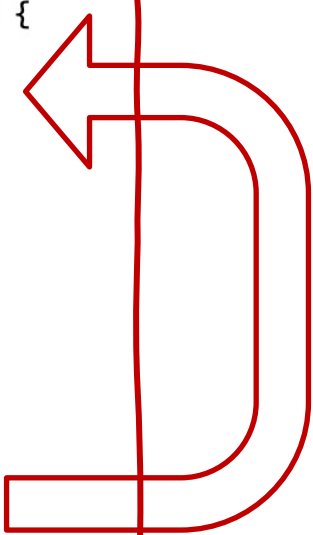
```
#include <iostream>

void scambiaByValue( double a, double b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByReference( double &a , double &b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByPointer( double* a , double* b ) {
    double c=*a;
    *a=*b;
    *b=c;
}

int main() {
    double a = 5 ;
    double b = 4 ;
    std::cout << a << " " << b << std::endl;
    scambiaByValue(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByReference(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByPointer(&a,&b) ;
}
```



Let's write functions to exchange the content of two variables.

❑ Functions can be placed before the main

- Passing by pointer: you pass the addresses of a and b in the main. The function works on addresses so the modifications in the function acts on the variables in the main

❑ Variables a and b in the main are exchanged

5. Functions : connections with the main

```
#include <iostream>

void scambiaByValue( double a, double b) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByReference( double &a , double &b ) {
    double temp = a ;
    a = b;
    b = temp;
}

void scambiaByPointer( double* a , double* b) {
    double c=*a;
    *a=*b;
    *b=c;
}

int main() {
    double a = 5 ;
    double b = 4 ;
    std::cout << a << " " << b << std::endl;
    scambiaByValue(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByReference(a,b) ;
    std::cout << a << " " << b << std::endl;
    scambiaByPointer(&a,&b) ;
}
```

Let's write functions to exchange the content of two variables.

❑ Functions can be placed before the main

• Passing by reference: the magic & tells the compiler to consider the variable in the main. In this case a and b inside the function are exactly the a and b in the main

- ❑ Variables a and b in the main are exchanged
- ❑ Works as for pointers but the syntax is easier !

Named entities, such as variables, functions, and compound types need to be declared before being used in C++. The point in the program where this declaration happens influences its visibility:

- ❑ An entity declared outside any block has *global scope*, meaning that its name is valid anywhere in the code (*global variables*).
- ❑ While an entity declared within a block, such as a function or a selective statement, has *block scope*, and is only visible within the specific block in which it is declared, but not outside it (*local variables*).

Intermezzo (1) : variables scope

cplusplus.com

Global variable

```
int glob;           // global variable
```

```
int some_function ()
```

```
{  
    int loc;         // local variable  
    loc = 0;  
}
```

local variable

```
int other_function ()
```

```
{  
    glob = 1; // ok: glob is a global variable  
    loc = 2; // wrong: loc is not visible from this function  
}
```

```
int main ()
```

```
{  
    glob = 1; // ok: glob is a global variable  
    loc = 2; // wrong: loc is not visible in the main  
}
```

```
> g++ -o main main.cpp  
main.cpp: In function 'int other_function()':  
main.cpp:12:3: error: 'loc' was not declared in this scope  
    loc = 2; // wrong: loc is not visible from this function  
    ^~~  
main.cpp:12:3: note: suggested alternative: 'glob'  
    loc = 2; // wrong: loc is not visible from this function  
    ^~~  
    glob  
main.cpp: In function 'int main()':  
main.cpp:18:3: error: 'loc' was not declared in this scope  
    loc = 2; // wrong: loc is not visible from this function  
    ^~~
```


Intermezzo (1) : variables scope

```
int glob;           // global variable

int some_function ()
{
    int loc;        // local variable
    loc = 0;
}

int other_function ()
{
    glob = 1; // ok: glob is a global variable
    loc = 2;  // wrong: loc is not visible from this function
}

int main ()
{
    glob = 1; // ok: glob is a global variable
    loc = 2;  // wrong: loc is not visible in the main
}
```



```
int glob;           // global variable

int some_function ()
{
    int loc;        // local variable
    loc = 0;
}

int other_function ()
{
    glob = 1; // ok: glob is a global variable
    int loc = 2; // ok: loc is a different local variable !
}

int main ()
{
    glob = 1; // ok: glob is a global variable
    loc = 2;  // wrong: loc is not visible in the main
}
```

Intermezzo (1) : variables scope

```
#include <iostream>
int glob = 2 ;

int main () {

    std::cout << "The value of glob is " << glob << std::endl;

    for ( int k = 0 ; k < 10 ; k++ ) {
        double x = double(k) + 3 ;
        std::cout << "The value of x is " << x << std::endl;
    }

    std::cout << "The value of x is " << x << std::endl; // WRONG !
}
```

x is a "local variable" that leaves inside the for loop only !

```
> g++ -o main main.cpp
main.cpp: In function 'int main()':
main.cpp:33:39: error: 'x' was not declared in this scope
    std::cout << "The value of x is " << x << std::endl;
                                      ^
```

Intermezzo (1) : variables scope, try to check with this !

Global variable : no scope, it can be accessed and modified everywhere

This variable is defined in the main, can be accessed from everywhere inside the main

This variable is defined in the for(){ }, can be accessed only there (doesn't exist outside)

```
#include <iostream>
using namespace std;
double var_global = 3 ;

void myFunction( double var_input ) {

    double var_infun = 2 ;
    cout << " Global = " << var_global
        << " input = " << var_input
        << " var_in_function = " << var_infun
        << " local1 = " << local1 << endl;
    << endl;
}

int main() {

    double var_local1 = 4 ;

    for ( int k = 0 ; k < 10 ; k++ ) {
        double var_local2 = k ;
        cout << "Global = " << var_global
            << " local1 = " << var_local1
            << " local2 = " << var_local2 << endl;
    }

    cout << "Global = " << var_global
        << " local1 = " << var_local1
        << " local2 = " << var_local2
        << " var_in_function = " << var_infun << endl;

    myFunction( var_local1 ) ;

}
```

Each variable lives inside the scope where it has been declared

These would cause a compilation error

Intermezzo (2) : what exactly std:: means ? Namespaces

Two variables with the same name in the same scope cannot exist. In each scope, a name can only represent one entity. For example, there cannot be two variables with the same name in the same scope

```
int some_function ()
{
    int x;
    x = 0;
    double x;    // wrong: name already used in this scope
    x = 0.0;
}
```

- ❑ This is seldom a problem for local names, since blocks tend to be relatively short, and names have particular purposes within them, such as naming a counter variable, an argument, etc...
- ❑ But non-local names bring more possibilities for name collision, especially considering that libraries may declare many functions, types, and variables, neither of them local in nature, and some of them very generic.
- ❑ Namespaces allow us to group named entities that otherwise would have *global scope* into narrower scopes, giving them *namespace scope*. This allows organizing the elements of programs into different logical scopes referred to by names.

```
namespace myNamespace
{
    int a, b;
}
```

Intermezzo (2) : what exactly std:: means ? Namespaces

Using namespaces, we can create two variables or member functions having the same name. Namespaces allow us to group named entities that otherwise would have *global scope* into narrower scopes, giving them *namespace scope*.

```
#include <iostream>

namespace rightmath {
    const int one = 1;
    const int two = 2;
    int add ( const int x, const int y ) { return x+y } ;
};

namespace wrongmath {
    const int one = 2;
    const int two = 1;
    int add ( const int x, const int y ) { return x-y } ;
};

using namespace rightmath;
using namespace std;

int main() {
    std::cout << "Addizione corretta :"  
        << rightmath::one << " + " << rightmath::two << " = " <<  
        << rightmath::add( rightmath::one , wrongmath::two ) << std::endl;
    cout << "Addizione sbagliata :"  
        << wrongmath::one << " + " << wrongmath::two << " = " <<  
        << add( wrongmath::one , wrongmath::two ) << std::endl;
}
```

The effect of the keyword
using : if not specified use
"rightmath" scope

6. Reading from and writing to files

```
#include <iostream>
#include <fstream>
```

<fstream> is needed to communicate with files

```
using namespace std;
```

```
int main() {
```

```
    ifstream inputFile("input.txt");
```

Create an ifstream variable called "inputFile" (constructor which accepts a file name as input) : this is used to read from file

```
    ofstream outputFile("output.txt");
```

Create an ofstream variable called "outputFile" (constructor which accept sa file name as input) : this is used to write to file

```
    if(!inputFile){
        cout <<"Error opening input file: exiting" <<endl;
        return -1;
    }
```

Check is the file is opened (good to use this check to avoid problems in the program execution)

```
    double a ;
```

```
    inputFile >>a;
    outputFile << a;
```

Read from and write to files

```
    inputFile.close();
    outputFile.close();
```

Closes the file currently associated with the object, disassociating it from the stream (not strictly necessary here as the destructor would do the same)

```
    return 0;
```

```
}
```

6. Reading from and writing to files

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {

    ifstream inputFile("input.txt") ;
    ofstream outputFile("output.txt") ;

    if(!inputFile){
        cout <<"Error opening input file: exiting" <<endl;
        return -1;
    }

    double a ;

    inputFile >>a;
    outputFile << a;

    inputFile.close();
    outputFile.close();

    return 0;
}
```

This program reads a number from a fixed file called "input.txt". Suppose I want to read numbers from different files, what should I do ?

- ☐ Open the code
- ☐ Modify the name of the input file (eg "input1.txt" or "input2.txt" ..)
- ☐ Recompile
- ☐ Run

Is there something a bit smarter I can do ? Try to parametrise the input filename (see next slides)

7. Program communication with outside : argv (argc)

argc is the size of the argv array
(determined by the executable)

```
#include <iostream>
#include <fstream>
#include <cstdlib>

using namespace std;

int main( int argc , char** argv ) {

    if ( argc < 3 ) {
        cout << "Uso del programma : " << argv[0] << " <ndati> <input_filename> <output_filename>" << endl;
        return -1;
    }

    cout << argc << endl;
    for ( int k = 0 ; k < argc ; k++ ) cout << argv[k] << endl;

    ifstream inputFile( argv[2] ) ;
    ofstream outputFile( argv[3] ) ;
    unsigned int ndati = atoi( argv[1] ) ;

    if(!inputFile){
        cout <<"Error opening input file: exiting" <<endl;
        return -1;
    } else {
        for ( int k = 0 ; k < ndati ; k++ ) {
            double dato ;
            inputFile >> dato;
            if ( inputFile.eof() ) {
                cout << "End of file reached, exiting" << endl;
                return -1;
            }
            cout << "Reading number " << dato <<endl;
            outputFile << dato << endl;
        }

        inputFile.close();
        outputFile.close();

        return 0 ;
    }
}
```

Accept inputs from outside

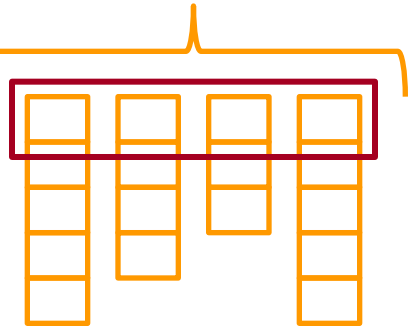
Check that all required arguments are there

Number of elements to read is passed as a char*, need to convert it into integer (use atoi from cstdlib)

Another useful check is if by chance the end of input file is reached (can't read more elements than the ones written in the file)

Return 0 means 'success', return any other number for 'failure'

argv



argv[0] nome dell'eseguibile

7. Program communication with outside : how it works

The diagram illustrates the workflow of a C++ program, showing the compilation and execution steps with annotations. The terminal output is as follows:

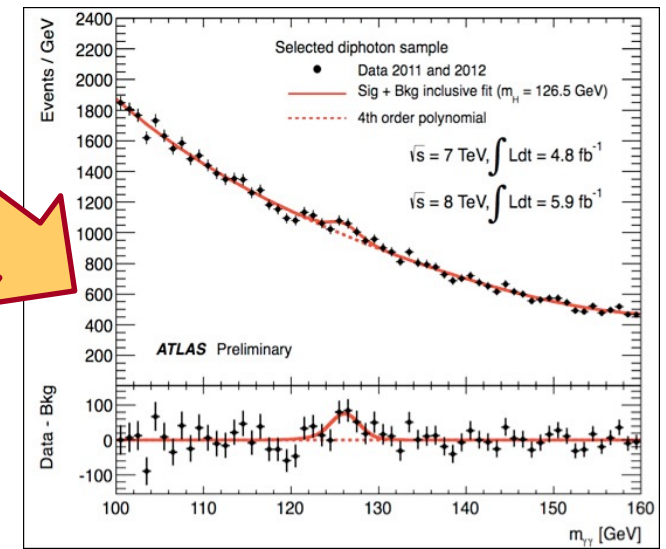
```
➤ g++ -o hw4 hw4.cpp
➤ ./hw4
Usò del programma : ./hw4 <ndati> <input_filename> <output_filename>
➤ ./hw4 10 inputdata.txt outputdata.txt
4
./hw4
10
inputdata.txt
outputdata.txt
Reading number 3
Reading number 4
Reading number 5
Reading number 7
Reading number 8
Reading number 9
End of file reached, exiting
```

Annotations and their corresponding actions:

- Compile your code**: Points to the compilation command `g++ -o hw4 hw4.cpp`.
- Launch the executable with no inputs**: Points to the command `./hw4`.
- The program reminds you how to use it if not launched correctly**: Points to the usage message `Usò del programma : ./hw4 <ndati> <input_filename> <output_filename>`.
- Now launch the executable with the right inputs**: Points to the command `./hw4 10 inputdata.txt outputdata.txt`.
- Enjoy the output !**: Points to the output lines `Reading number 3` through `Reading number 9`.

The problem

```
1935.39 1934.7 1935.21 1934.29 1934.19 1933.48 1933.6 1932.97 1933.05 1932.42 1932.35 1933.01 1936.87 1934.33 1933.4
1931.28 1931.42 1931.92 1933.15 1935.88 1938.46 1939.86 1941.86 1942.48 1943.67 1947.13 1947.45 1949.35 1950.84 1949
1960.46 1962.84 1964.56 1966.22 1966.11 1971.99 1972.51 1977.67 1978.36 1982.35 1983.89 1989.56 1991.81 1994.29 1997
1974.2 2003.67 2009.42 2014.28 2023.67 2024.28 2025.65 2014.25 2010.06 2028.24 2014.78 2010.49 2001.44 1997.32 2001.
1994.95 1985.42 1983.57 1989.46 1987.63 1990.82 1986.2 1987.84 1993.8 1994.39 1994.75 1995.29 1998.68 2002.47 1997.5
2006.69 2007.57 2009.82 2010.84 2011.65 2011.26 2010.85 2009.87 2012.45 2010.43 2012.87 2011.85 2009.57 2010.81 2011
2014.52 2013.68 2016.1 2015.88 2015.08 2016.6 2016.04 2017.69 2017.91 2019.4 2020.1 2022.61 2023.88 2026.96 2028.45
2035.61 2033.01 2035.36 2036.29 2034.13 2038.89 2041.37 2040.18 2038.72 2040.36 2042.09 2044.33 2045.48 2046.81 2047
2042.53 2041.79 2042.12 2040.69 2040.42 2040.33 2041.49 2038.23 2038.44 2040.31 2037.93 2040.33 2039.88 2036.79 2035
2034.76 2035.65 2034.63 2036.12 2034 2033.51 2032.18 2032.54 2033.55 2032.65 2031.92 2031.87 2030.88 2031.14 2030.12
2029.45 2030.02 2030.28 2030.13 2030.43 2030.39 2030.69 2030.48 2030.12 2030.24 2030.19 2030.2 2030.07 2029.99 2029.
2029.25 2029.04 2029.02 2028.75 2028.54 2028.66 2028.53 2028.26 2028.21 2026.52 2027.31 2028.04 2027.96 2027.64 2027
2026.03 2026.15 2027.24 2026 2027.76 2027.42 2027.92 2028.2 2026.66 2025.47 2027.37 2023.62 2023.7 2023.72 2024.07 2
2024.71 2024.69 2024.68 2024.79 2024.51 2024.33 2024.25 2024.2 2023.74 2023.78 2023.81 2024.2 2026.67 2026.75 2027.5
2027.04 2027.39 2027.13 2027.26 2026.93 2027.63 2027.18 2026.82 2025.25 2024.38 2023.83 2023.71 2024.05 2023.27 2023
2023.53 2024.72 2026.82 2026.54 2027.04 2025.32 2025.74 2025.81 2026.99 2026.75 2026.25 2027.23 2027.05 2028
2028.41 2028.85 2028.08 2028.3 2029.74 2030.44 2031.23 2033.31 2033.51 2032.49 2034.8 2032.6 2033.45
2028.73 2027.15 2023.94 2025.08 2025.21 2025.28 2023.94 2025.71 2022.91 2021.59 2025.2
2026.48 2033.23 2035.23 2036.88 2035.36 2033.88 2030.99 2030.83 2031.23 2029.01 2029.2
2024.1 2026.51 2023.04 2023.35 2023.62 2022.84 2022.82 2021.5 2020.7 2021.69 2022.39 2019
2014.46 2010.17 2008.09 2008.47 2009.86 2010.48 2015.74 2009.95 2007.59 2008.58 2010.78 2004
1999.82 1998.05 1997.74 2001.86 2002.05 1997.57 1997.62 1995.71 1994.33 1994.74 1990.25 1990.82
1985.28 1985.28 1981.63 1984.68 1984.73 1982.88 1982.22 1985.36 1984.17 1985.3 1986.17 2006.84 1986
1973.17 1971.89 1971.19 1975.18 1985.57 1981.53 1981.41 1978.42 1973.22 1970.43 1968.6 1965.03 1964.2
1958.69 1958.97 1957.35 1957.84 1955.22 1954.39 1953.1 1952.52 1950.38 1949.66 1949.92 1947.94 1950.53
1943.05 1942.3 1942.8 1941.81 1941.56 1941.1 1941.02 1939.32 1938.52 1938.2 1937.92 1936.52 1936.4 1935
1935.75 1935.29 1934.96 1935.85 1937.71 1939.46 1939.74 1935.9 1937.51 1934.89 1932.98 1931.02 1931.32
1926.15 1925.21 1924.92 1923.98 1922.78 1921.77 1921.51 1920.14 1919.55 1918.74 1919.03 1917.8 1917.49
1913.46 1913.03 1912.09 1912.02 1911.04 1910.25 1909.77 1909.41 1908.65 1907.4 1906.47 1905.49 1906 1904
1898.46 1897.82 1896.23 1896.16 1893.56 1893.92 1892.23 1891.86 1892.3 1892.94 1892.45 1891.68 1891.86
```



Our goal in the first 3 lessons will be to analyze a dataset (a bunch of numbers)

- ❑ Data container
- ❑ Operations on data

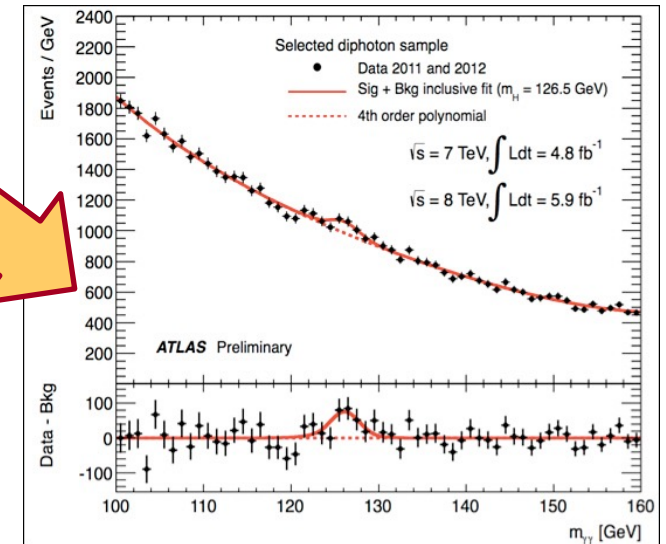
Average, variance, median

The problem

```
1935.39 1934.7 1935.21 1934.29 1934.19 1933.48 1933.6 1932.97 1933.05 1932.42 1932.35 1933.01 1936.87 1934.33 1933.4
1931.28 1931.42 1931.92 1933.15 1935.88 1938.46 1939.86 1941.86 1942.48 1943.67 1947.13 1947.45 1949.35 1950.84 1949
1960.46 1962.84 1964.56 1966.22 1966.11 1971.99 1972.51 1977.67 1978.36 1982.35 1983.89 1989.56 1991.81 1994.29 1997
1974.2 2003.67 2009.42 2014.28 2023.67 2024.28 2025.65 2014.25 2010.06 2028.24 2014.78 2010.49 2001.44 1997.32 2001.
1994.95 1985.42 1983.57 1989.46 1987.63 1990.82 1986.2 1987.84 1993.8 1994.39 1994.75 1995.29 1998.68 2002.47 1997.5
2006.69 2007.57 2009.82 2010.84 2011.65 2011.26 2010.85 2009.87 2012.45 2010.43 2012.87 2011.85 2009.57 2010.81 2011
2014.52 2013.68 2016.1 2015.88 2015.08 2016.6 2016.04 2017.69 2017.91 2019.4 2020.1 2022.61 2023.88 2026.96 2028.45
2035.61 2033.01 2035.36 2036.29 2034.13 2038.89 2041.37 2040.18 2038.72 2040.36 2042.09 2044.33 2045.48 2046.81 2047
2042.53 2041.79 2042.12 2040.69 2040.42 2040.33 2041.49 2038.23 2038.44 2040.31 2037.93 2040.33 2039.88 2036.79 2035
2034.76 2035.65 2034.63 2036.12 2034 2033.51 2032.18 2032.54 2033.55 2032.65 2031.92 2031.87 2030.88 2031.14 2030.12
2029.45 2030.02 2030.28 2030.13 2030.43 2030.39 2030.69 2030.48 2030.12 2030.24 2030.19 2030.2 2030.07 2029.99 2029.
2029.25 2029.04 2029.02 2028.75 2028.54 2028.66 2028.53 2028.26 2028.21 2026.52 2027.31 2028.04 2027.96 2027.64 2027
2026.03 2026.15 2027.24 2026 2027.76 2027.42 2027.92 2028.2 2026.66 2025.47 2027.37 2023.62 2023.7 2023.72 2024.07 2
2024.71 2024.69 2024.68 2024.79 2024.51 2024.33 2024.25 2024.2 2023.74 2023.78 2023.81 2024.2 2026.67 2026.75 2027.5
2027.04 2027.39 2027.13 2027.26 2026.93 2027.63 2027.18 2026.82 2025.25 2024.38 2023.83 2023.71 2024.05 2023.27 2023
2023.53 2024.72 2026.82 2026.54 2027.04 2025.32 2025.74 2025.81 2026.99 2026.75 2026.25 2026.23 2027.23 2027.05 2028
2028.41 2028.85 2028.08 2028.3 2029.74 2030.44 2031.23 2033.31 2033.51 2032.49 2034.8 2034.7 2032.6 2033.45
2028.73 2027.15 2023.94 2025.08 2025.21 2025.28 2023.94 2025.71 2022.91 2021.59 2025.2 2024.7 2034.65 2037
2026.48 2033.23 2035.23 2036.88 2035.36 2033.88 2030.99 2030.83 2031.23 2029.01 2029.2 2028.51 2029
2024.1 2026.51 2023.04 2023.35 2023.62 2022.84 2022.82 2021.5 2020.7 2021.69 2022.39 2019.7 2016.87
2014.46 2010.17 2008.09 2008.47 2009.06 2010.48 2015.74 2009.95 2007.59 2008.58 2010.78 2004.
1999.82 1998.05 1997.74 2001.86 2002.05 1997.57 1997.62 1995.71 1994.33 1994.74 1990.25 1990.82
1985.28 1985.28 1981.63 1984.68 1984.73 1982.88 1982.22 1985.36 1984.17 1985.3 1986.17 2006.84 1986
1973.17 1971.89 1971.19 1975.18 1985.57 1981.53 1981.41 1978.42 1973.22 1970.43 1968.6 1965.03 1964.2
1958.69 1958.97 1957.35 1957.84 1955.22 1954.39 1953.1 1952.52 1950.38 1949.66 1949.92 1947.94 1950.53
1943.05 1942.3 1942.8 1941.81 1941.56 1941.1 1941.02 1939.32 1938.52 1938.2 1937.92 1936.52 1936.4 1935
1935.75 1935.29 1934.96 1935.85 1937.71 1939.46 1939.74 1935.9 1937.51 1934.89 1932.98 1931.02 1931.32
1926.15 1925.21 1924.92 1923.98 1922.78 1921.77 1921.51 1920.14 1919.55 1918.74 1919.03 1917.8 1917.49
1913.46 1913.03 1912.09 1912.02 1911.04 1910.25 1909.77 1909.41 1908.65 1907.4 1906.47 1905.49 1906 1904
1898.46 1897.82 1896.23 1896.16 1893.56 1893.92 1892.23 1891.86 1892.3 1892.94 1892.45 1891.68 1891.86
```



Lesson 1 : read a **given number** of elements from a file. The number of elements to be read **is passed by the user at runtime** (not compilation time !)



Average, variance, median

The most naïve container

```
#include <iostream>
#include <cstdlib>

using namespace std;

int main( int argc , char** argv) {

    if ( argc < 2 ) {
        cout << "Uso del programma : " << argv[0] << " <nelements>" << endl;
        return -1;
    }

    int nelements = atoi(argv[1]);

    double vfixed[nelements] ;

    for ( int k = 0 ; k < nelements ; k++ ) {
        vfixed[k] = k*k ;
    }

    double * vdynamic = new double[nelements] ;

    for ( int k = 0 ; k < nelements ; k++ ) {
        vdynamic[k] = k*k ;
    }

    for ( int k = 0 ; k < nelements ; k++ ) {
        cout << "Component k = " << vdynamic[k] << endl;
    }

    delete [] vdynamic ;

    return 0 ;
}
```

vfixed has a size which is set at compilation time : set to what ? To the content of nelements which is undefined.
This will create an array of undefined size !

vdynamic has a size which is set at runtime : to your exact desired value nelements which is correctly filled at execution time

don't forget to release the allocated memory (at this point is not that important because we are close to the end of the program)

Memory leak !

Suppose you need to loop over 1M files, read a bunch of numbers from each files

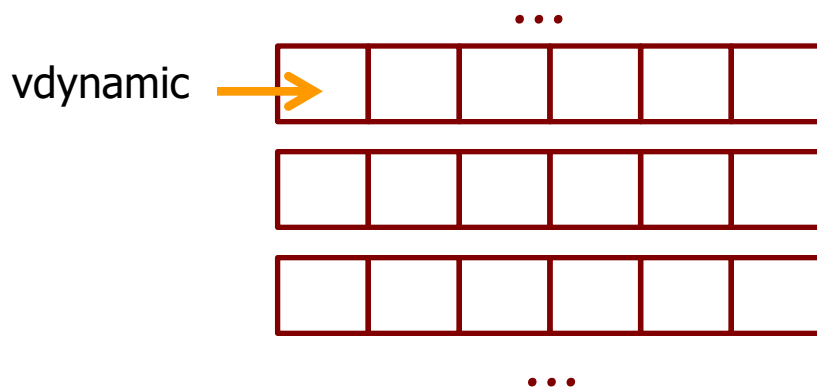
```
for ( int nfiles = 0 ; nfiles < 1000000 ; nfiles++ ) {  
    ifstream fin( filenames[k] );  
  
    double * vdynamic = new double[nelements] ;  
    double dato;  
  
    for ( int k = 0 ; k < nelements ; k++ ) {  
        fin >> dato;  
        vdynamic[k] = data;  
    } ;  
    delete vdynamic ;  
}
```

Loop on the list of files

Allocate dynamically a data container

Fill the container

Don't forget to clean up memory !



- ❑ vdynamic is a pointer to the first element of the array
- ❑ When the pointer goes out of scope the pointer is cancelled but the allocated memory is not ! Memory allocated for the array is declared as filled and there's no way to access it anymore
- ❑ So, each new pointer will keep allocating new memory and leave it occupied
- ❑ Memory leak : the program will crash when the physical memory of the machine is completely filled
- ❑ Use smart pointers !

1. So let's go and setup our first program

Have a look at esercizio1.0.cpp

1. Open a data file, read the numbers and put them into an array
2. Compute the mean and the standard deviation of the numbers in the array
3. Order the numbers in the array and compute the median
4. Write ordered array into a file

then compile in this way

```
g++ esercizio1.0.cpp -o esercizio1.0
```

Intermezzo (III) : compilation

Compilation can be broken into three main parts

1. pre-processing

2. actual compilation

3. linking

It is the phase in which the directives to the compiler are managed:

- #Include expansion: the compiler loads the declarations of the functions defined in the included libraries in order to check that these functions are used correctly in the program
 - Replacing constants defined with #define: the compiler searches for all occurrences of these constants in the program and replaces them with the corresponding values
- Management of other directives that we will not see ...

Try `gcc -E esercizio1.0.cpp`

Intermezzo (III) : compilation

Compilation can be broken into three main parts

1. pre-processing

2. actual compilation

3. linking

It is the phase in which the program is transformed into binary code (not yet in an executable). It consists of several sub-phases (not necessarily sequential):

- Type control check that the variables used have been declared, check that operations in expressions are used with arguments of an appropriate type, check that the functions are called with number and type parameters appropriate...
- Analysis and optimizations : elimination of dead code, optimisation of the code...
- Generation of the binary code

Try `g++ -c esercizio1.0.cpp`

Intermezzo (III) : compilation

Compilation can be broken into three main parts

1. pre-processing

2. actual compilation

3. linking

Multiple object modules are linked together to create an executable file

- file1.o file2.o ...: different modules object of the same program obtained through separate compilation (we will see shortly)
- standard and system libraries
- libfile1.a libfile2.a ...: several external libraries that provide functions used within the program The result of this phase is a single executable file

Try `g++ -o esercizio esercizio1.0.cpp`

2. Try to make the program a bit easier to read : esercizio1.1.cpp (split into functions)

```
#include <iostream>
#include <fstream>
#include <cstdlib>

using namespace std;

double CalcolaMedia( double * , int ) ;
double * ReadDataFromFile ( const char* , int ) ;
void Print ( const char* , double * , int ) ;

int main ( int argc , char** argv ) {

    if ( argc < 2 ) {
        cout << "Uso del programma : " << argv[0] << " <n_data> <filename> " << endl;
        return -1 ;
    }

    int ndata = atoi(argv[1]);
    char * filename = argv[2];

    double * data = ReadDataFromFile ( filename, ndata ) ;
    cout << "Media = " << CalcolaMedia( data , ndata ) << endl;
    Print( "fileout.txt", data, ndata ) ;

}

double * ReadDataFromFile ( const char* Filename , int size ) {
    double * data = new double[size];
    // [ ... ]
    return data;
}

void Print ( const char* Filename, double * data, int size ) {
    // [ ... ]
}

double CalcolaMedia( double * data , int size ) {
    // [ ... ]
    return media ;
}
```

Functions declarations: input and return type only are important at this stage

The main simply calls the functions

Functions implementation : code how the function should work

2. Try to make the program a bit easier to read : esercizio1.1.cpp (split into functions)

```
#include <iostream>
#include <fstream>
#include <cstdlib>

using namespace std;

double CalcolaMedia( double * , int ) ;
double * ReadDataFromFile ( const char* , int ) ;
void Print ( const char* , double * , int ) ;

int main ( int argc , char** argv) {

    if ( argc < 2 ) {
        cout << "Uso del programma : " << argv[0] << " <n_data> <filename> " << endl;
        return -1 ;
    }
```

```
int n_data;
char * filename;
```

```
double * data;
```

```
cout << "Inserisci il numero di dati da leggere : ";
```

```
Print( "Inserisci il file da leggere : ", data, n_data );
```

```
double * ReadDataFromFile ( const char* Filename , int size ) {
    double * data = new double[size];
    // [ ... ]
    return data;
}
```

```
void Print ( const char* Filename, double * data, int size ) {
    // [ ... ]
}
```

```
double CalcolaMedia( double * data , int size ) {
    // [ ... ]
    return media ;
}
```

then compile in this way

g++ esercizio1.1.cpp -o esercizio1.1

3. Can we go further (esercizio1.2.cpp) ?

```
#include <iostream>
#include <fstream>
#include <cstdlib>

using namespace std;

double CalcolaMedia( double * , int ) ;
double * ReadDataFromFile ( const char* , int ) ;
void Print ( const char* , double * , int ) ;

int main ( int argc , char** argv ) {

    if ( argc < 2 ) {
        cout << "Uso del programma : " << argv[0] << " <n_data> <filename> " << endl;
        return -1 ;
    }

    int ndata = atoi(argv[1]);
    char * filename = argv[2];

    double * data = ReadDataFromFile ( filename, ndata ) ;
    cout << "Media = " << CalcolaMedia( data , ndata ) << endl;
    Print( "fileout.txt", data, ndata ) ;

}

double * ReadDataFromFile ( const char* Filename , int size ) {
    double * data = new double[size];
    // [ ... ]
    return data;
}

void Print ( const char* Filename, double * data, int size ) {
    // [ ... ]
}

double CalcolaMedia( double * data , int size ) {
    // [ ... ]
    return media ;
}
```

Move the functions declarations into a dedicated file (header file, funzioni.h) and replace this with `#include "funzioni.h"`

The main doesn't change

Move the functions implementation into a dedicated file (funzioni.cpp or funzioni.cxx)

4. Can we go further (esercizio1.2.cpp) ?

```
#include <iostream>
#include <fstream>
#include <cstdlib>

#include "funzioni.h" ←

using namespace std;

int main ( int argc , char** argv ) {

    if ( argc < 2 ) {
        cout << "Uso del programma : " << argv[0] << " <n_data> <filename> " << endl;
        return -1 ;
    }

    int ndata = atoi(argv[1]);
    char * filename = argv[2];

    double * data = ReadDataFromFile ( filename, ndata ) ;
    cout << "Media = " << CalcolaMedia( data , ndata ) << endl;
    Print( "fileout.txt", data, ndata ) ;

}
```

Move the functions declarations into a dedicated file (header file, funzioni.h) and replace this with `#include "funzioni.h"`

then compile in this way

```
g++ -o esercizio1.1 esercizio1.1.cpp funzioni.cpp
```

Makefile

The makefile helps you in organizing the compilation instructions: just create a new file called Makefile

```
esercizio1.2 : esercizio1.2.cpp funzioni.cpp funzioni.h
    g++ esercizio1.2.cpp funzioni.cpp -o esercizio1.2

clean:
    rm esercizio1.2
```

❑ Structure of the makefile

<target> : <dep1> <dep2> <dep3> ...
[tab] <compilation instruction>

```
esercizio1.2 : esercizio1.2.o funzioni.o
    g++ esercizio1.2.o funzioni.o -o esercizio1.2

funzioni.o: funzioni.cpp funzioni.h
    g++ -c funzioni.cpp -o funzioni.o

esercizio1.2.o : esercizio1.2.cpp funzioni.h
    g++ -c esercizio1.2.cpp -o esercizio1.2.o

clean:
    rm esercizio1.2
    rm *.o
```

- ❑ The name of the target must match the name of the output file
- ❑ Dependencies : list of files that needs to be checked to trigger a new compilation. The timestamps of the dependencies are compared to the timestamp of the output file.
- ❑ The option `-c` forces `g++` to compile but not link
- ❑ To launch a compilation just type "make esercizio1.2" (if you simply write make it will execute the first target)

Only the parts of the programs which have been modified are recompiled !

Makefile

Some useful parameters of the gcc compiler are as follows:

- `-o`: Allows you to specify the name of the executable file to be generated (as an alternative to `a.out`).
 - Usage: `g++ -o esercizio1.0 esercizio1.0.cpp ( g++ esercizio1.0.cpp -o esercizio1.0 )`
- `-Wall`: Enables the display of all warning messages generated during compilation
- `-pedantic`: Check that the program exactly meets the rules of the C standard (ISO C). Report any violations of the standard with warning messages
- `-c` : compilation but no linking

It is always good to use gcc at least as follows (might need to add `-c` if needed):

```
g++ -Wall -o esercizio1.1 esercizio1.1.cpp funzioni.cpp
```

What's in today menu

Take away messages from today lecture (in random order):

- ❑ What a namespace is
- ❑ Variables scope
- ❑ How to pass inputs to your executable (argv,argc)
- ❑ Fixed size array (at compilation level) vs variable size array (at run-time)
- ❑ Break the code into functions, how variables enter (reference, value and pointer) and how they exit. Break into different files accordingly
- ❑ Read/write to files
- ❑ Compilation and makefiles

More variants

```
if(!inputFile){
    cout <<"Error opening input file: exiting" <<endl;
    return -1;
} else {
    for ( int k = 0 ; k < ndati ; k++ ) {
        double dato ;
        inputFile >> dato;
        if (inputFile.eof() ) {
            cout << "End of file reached, exiting" << endl;
            return -1;
        }
        cout << "Reading from input file :" << dato << endl;
        outputFile << dato << endl;
    }
}
```

```
int nelements = 0;
if(!inputFile){
    cout <<"Error opening input file: exiting" <<endl;
    return -1;
} else {
    while (true) {
        double dato;
        inputFile >> dato;
        if( inputFile.eof() ) break;
        nelements++;
        cout << "Reading from input file :" << dato << endl;
        outputFile << dato << endl;
    };
};
```